

Performance Model Calculator for Basketball Players

PJ Neidlinger

Fall 2018

Senior Project Final Report:

Performance Metric for Basketball Players

Student Name: PJ Neidlinger

Advisor Name: Dr. Sean Hayes

Expected Date of Graduation: December 2018

Description of Project: This project is a basketball statistics program that will be able to compute statistics for basketball of all levels. Using a player's points, rebounds, assists, steals, blocks, three pointers made, turnovers, field goals made, field goals attempted, free throws made, and free throws attempted, the program will generate a single value that will be a representation of how well the player performed for a game or set of games. This project will be a fully functional web application that users will be able to visit on the internet. The formula that will be used is as follows:

$$\begin{aligned} & (\text{Points Value} / \text{Points Value} \times \text{Points Input}) + \\ & (\text{Points Value} / \text{Rebounds Value} \times \text{Rebounds Input}) + \\ & (\text{Points Value} / \text{Assists Value} \times \text{Assists Input}) + \\ & (\text{Points Value} / \text{Steals Value} \times \text{Steals Input}) + \\ & (\text{Points Value} / \text{Blocks Value} \times \text{Blocks Input}) + \\ & (\text{Points Value} / \text{Threes Value} \times \text{Threes Input}) + \\ & (\text{Points Value} / \text{Field Goals Attempted Value} \times \text{Field Goals Attempted Input}) \times (((\text{Input} \\ & \text{Field Goals Made} / \text{Input Field Goals Attempted}) - \text{Field Goal Percentage Value}) / \text{Field} \\ & \text{Goal Percentage Value}) + \\ & (\text{Points Value} / \text{Free Throws Attempted Value} \times \text{Free Throws Attempted Input}) \times \\ & (((\text{Input Free Throws Made} / \text{Input Free Throws Attempted}) - \text{Free Throw Percentage} \\ & \text{Value}) / \text{Free Throw Percentage Value}) - \\ & (\text{Points Value} / \text{Turnovers Value} \times \text{Turnovers Input}) \end{aligned}$$

The statistics that are listed as “Value” are values that will be pulled from the statistical leaderboards, and the statistics that are listed as “Input” will be the values that correspond to a player that is being evaluated.

Implementation Language(s): The final version of this program will be implemented in Python for all aspects. Within Python, the Requests library and the Flask microframework will be used to implement the program. There will also be some instances of HTML to establish and set up some aspects of the web pages. This program will be deployed as a working and fully accessible web application to Heroku.

Any software / hardware equipment needed: This program will be developed using the *vi* text editor within the Linux command line to develop many aspects of the program. Along with this, there will be situations where the Atom and Visual Studio Code development environments are used to simplify and streamline development. Also within development, VirtualBox will be used to have access to and develop the program while away from a desktop computer. A nominal computer environment with access to the internet will be sufficient for any user to access the program.

Motivation and Problem Statement: I am an avid basketball fan, and I have participated in fantasy basketball leagues for many of the recent years. These leagues require participants to draft a team of National Basketball Association players, and the participant whose drafted team performs the best will win the league. When trying to decide which players to draft or trade, there are many statistics to take into consideration. The statistics of points, rebounds, assists, steals, blocks, three pointers made, turnovers, field goal percentage, and free throw percentage are all statistics that each player has that contribute to a team’s performance. Ideally, there would

be a simple quantifiable way to determine if one player is better than another overall. The current method of comparing nine values of one player to nine values of another can be problematic, because some players specialize in one statistic. If one player is extremely good at scoring points, and another is extremely good at obtaining rebounds and blocks, it is difficult to determine which player will most positively affect a team. I have developed a formula that gives each statistic weight based on an above average value for that statistic, determined from the online National Basketball Association leaderboards that are updated after every game that is played. This formula gives a weight to and combines each statistic into one value that can be used to determine how well a player has performed. This single number, within which a higher value corresponds to better performance, drastically simplifies player statistics from nine comparable values to one comparable value. This will make it easier for users to compare players they are debating on drafting or trading for. Along with this, outside of the fantasy basketball spectrum, a player's real value can be more easily assessed, as this technology could be implemented within the National Basketball Association to determine player value and performance.

System Requirements: The basic functionality of this program is to take statistical input from a user and combine the data into one value that represents how good the statistics are. To do this, a few features are required. First, the program must be able to pull the target values from the National Basketball Association statistical leaderboards. Following this, the program will need to be able to store the pulled data in a "data.txt" file in the correct format. Next, the program will need to read in the data from the previously created file, in order to use it in computation of the metric. Following this, the program must be able to take input from a user. This input will be eleven values that will represent a basketball player's statistics for a game or set of games.

Within this requirement, the program must ensure that valid and usable data is entered by the user. Next, the program needs to be able to calculate the performance metric based on the pulled leaderboard values and the input from the user. This metric will be the result of the aforementioned formula. Finally, the program must be able to display the metric back to the user. Any metrics that are produced via a user will be stored and displayed in descending order until the page is reloaded. The system will also be deployed to Heroku and therefore available to users with access to the internet and the page URL.

User Interface Design:

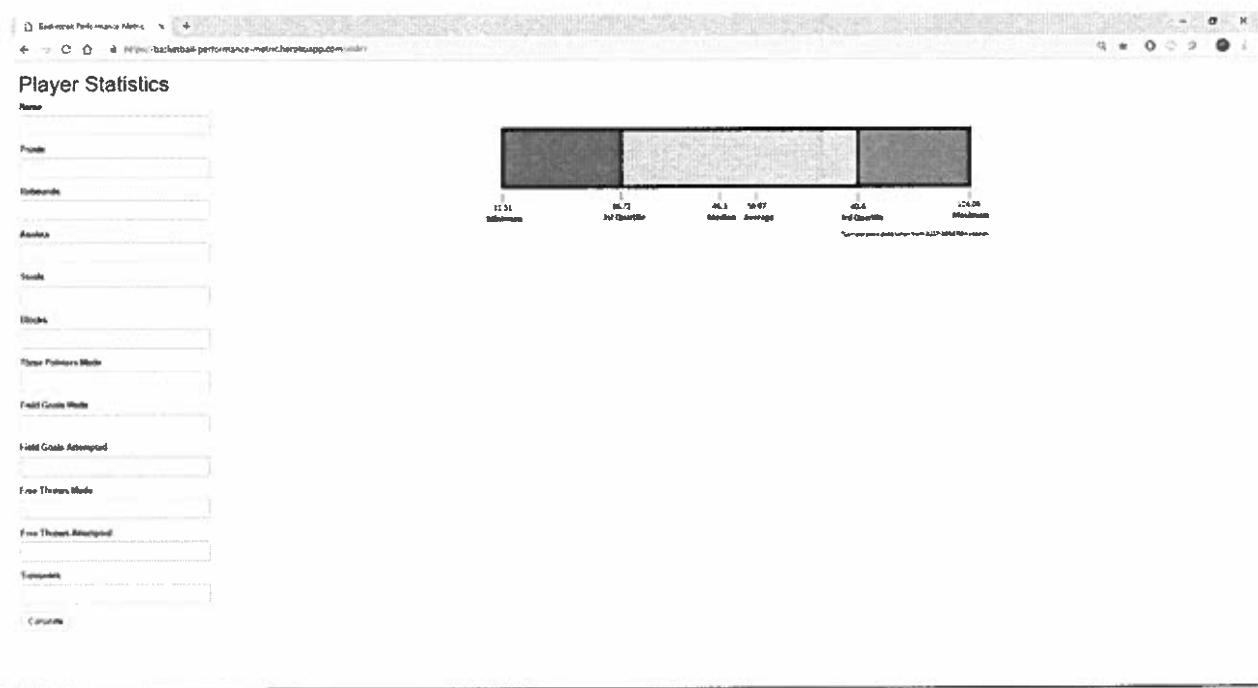


Figure 1: Base page when loaded

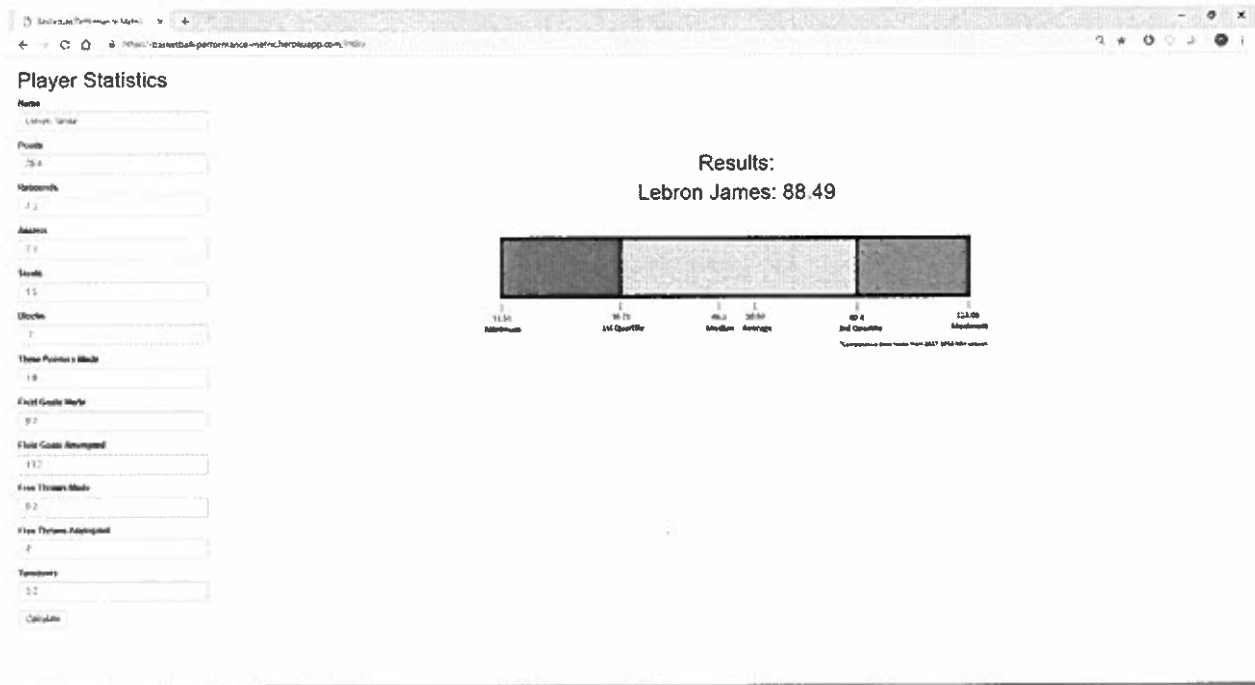


Figure 2: Page after one player has been evaluated

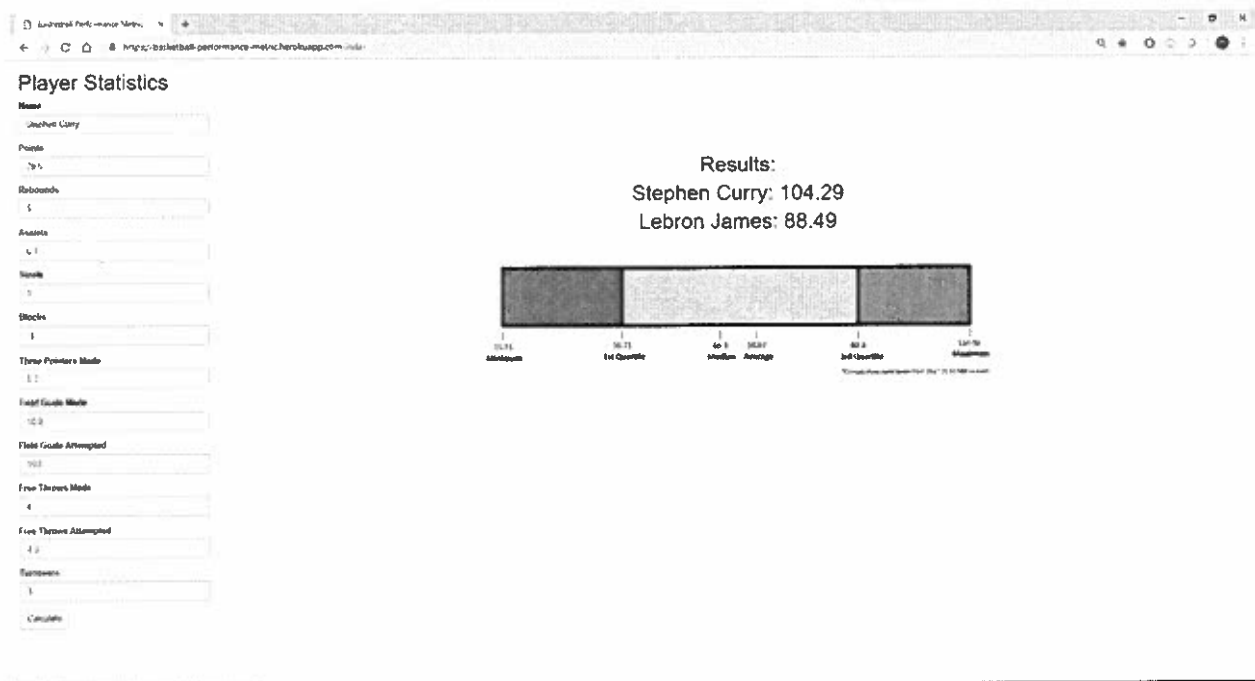


Figure 3: Page after two players have been evaluated

Test Plan Identifier:

- Performance Metric for Basketball Players Test Plan

Introduction:

- Testing will be performed to ensure that the product works as desired
- Testing will be done to prevent errors within the program
- Testing will need to be done by December 2018, to ensure a working, tested product is delivered on time

References:

- Project Proposal

Test Items:

- Program will be developed in the Linux subsystem on Windows 10 version 1809, along with a Linux virtual environment through the use of the VirtualBox software
- Testing will be done by exhausting any possible forms of failure to ensure all possible errors are covered and prevented to the best possible standard

Features to be Tested:

- Pulling data from the National Basketball Association leaderboards
- Storing the necessary data in a usable manner
- Reading the data
- Taking input from the user
- Calculating metrics using the data and the user input
- Outputting the metrics in the necessary format

Approach:

- The overall approach will be to test the product by interacting with and giving all possible forms of failure to each individual component, to ensure that all errors are covered
- If an error, like an input error, is encountered, the product should be able to handle the error without something catastrophic happening
- The product, after testing, will be able to be a standalone product that can encounter errors and inform the user on what has happened and what they should do

Item Pass/Fail Criteria:

- An item will pass testing if it can encounter an error, give a correct error message, inform the user what has happened, and inform the user what they should next do
- An item will fail testing if it encounters an error, and causes the application to stop or freeze or encounter a catastrophic error

Test Deliverables:

- Test Plan
- Test Cases
- Defect/Enhancement Logs
- Test Reports

Test Environment:

- Testing will be performed in the Linux subsystem on Windows 10 version 1809, a virtual environment via VirtualBox, and in the web application itself

Estimate:

- Testing will not cost anything of monetary value, other than the time it takes to do so
- Testing will be given a two-week period after the conclusion of the development of the product, and will be done before the delivery of the product

Schedule:

- There are six individual items that will need to be tested that are listed above, and they will be tested in the above order
- The program will need to do these things in the order given, and so they will be tested in that order
- There will be six milestones:
 - Conclusion of the testing for pulling data from the National Basketball Association Leaderboards
 - Conclusion of the testing for storing the necessary data in a usable manner
 - Conclusion of the testing for reading the data
 - Conclusion of the testing for taking input from the user
 - Conclusion of the testing for calculating metrics using the data and the user input
 - Conclusion of the testing for outputting the metrics in the necessary format

Responsibilities:

- PJ Neidlinger will be responsible for performing all of the testing, including the testing of all six items

Risks:

- The main risk for this product would be that, if an error is run into, the program would stop working and enter a state where it is unusable
- To prevent this from happening, testing will be performed to ensure that the product is continually usable and user-friendly at all times

Approvals:

- Dr. Sean Hayes

Test Results: The first two features, pulling data from the National Basketball Association leaderboards and storing the necessary data in a usable manner, were both tested and error checked without any issues. Testing was done to ensure that values are successfully found and correctly written to the data file, and no issues were found. The third feature, reading the data, did produce an issue. When reading the file in the Python routes file, a mistake was made, and the file was not closed after reading. This issue was found and corrected. The next feature, taking input from the user, was tested with no issues. Validators within the Flask framework have been used to ensure that reasonable numbers are entered in the statistics fields, and so therefore problematic data cannot be entered. The fifth feature, calculating metrics using the data and the user input, did encounter an issue when tested. The only dividing that happens with user data is when the field goal and free throw percentages are calculated using the input made divided by the input attempted. There was a mistake where error checking to see whether the attempted value was zero was not implemented. Because of this, if a user entered statistics where a player attempted either or both of zero field goals or zero free throws, the application would try to divide by zero and enter an error state. This was discovered during testing and remedied by implementing a check to see whether the value is zero before trying to divide by the value. If the value is zero, the divide does not occur. The final feature, outputting the metrics in the necessary format, was tested with no resulting issues. The storing of and sorting of previous values works as desired after the completion of the implementation.

Future Work / Areas of Improvement: As far as the user interface is concerned, I would like to look into the possibility of having a table of players already populated on the webpage for users to interact with. This table could list all National Basketball Association players and their statistics, and simply allow users to select rows to see the performance metric for those players.

This would streamline the process of users obtaining the data they are looking for and prevent users from having to type in eleven values. Also in the future, I would like to evaluate the predictive properties of the metric. For example, if a player was averaging a 42.2 performance metric over the first thirty games of the season, but over the next ten games he averages a 51.6 performance metric, he might be someone that a fantasy basketball player could look into claiming. The application could potentially alert a user of a player that is trending upward and that they should look into obtaining. This would be a positive addition to the functionality and would increase the value of the application.

Basketball Performance Metric Program

By: PJ Neidlinger

Faculty Advisor: Dr. Sean Hayes

Problem Statement

- How to choose between multiple fantasy basketball players who are good in opposing statistics
- Example:

Points	Rebounds	Assists	Steals	Blocks	Threes	Field Goal %	Free Throw %	Turnovers
23	3.9	5.2	1.1	.3	1.1	45.8%	82.9%	2.2
13.9	10.8	.9	.8	1.9	0	66%	54.3%	1.4

- Due to players having opposing talents, an individual is often left to guess

Background: Fantasy Basketball

- League where individuals populate a fantasy team with real NBA players
- Each players' performance in real games affect their fantasy team's success
- Individuals initially add players to their team in a league draft
- Able to trade players to other teams in the league
- Able to drop a player from a team and pick up a player that is not on a team
- Individuals hope to choose the player that will most positively affect their team in all three circumstances

Solution

- Compute one number that represents how good a player is
- Done by weighting each statistic based on how rare each is
- Weighted by first pulling current NBA values when the page is loaded
- Calculating weights is done based on pulled values
- Then multiplying each simple statistic by that statistic's weight
- Percentage values are then multiplied by their percent error

Solution

- Points, rebounds, assists, steals, blocks, and threes add points to a player's metric
- Turnovers subtract points from a player's metric
- Field and free throw percentages add points to a player's metric if the percentage is above the target value, and subtract points if the percentage is below the target value

Formula

$$\begin{aligned}
 & (\text{Points Value} / \text{Points Value} \times \text{Points Input}) + \\
 & (\text{Points Value} / \text{Rebounds Value} \times \text{Rebounds Input}) + \\
 & (\text{Points Value} / \text{Assists Value} \times \text{Assists Input}) + \\
 & (\text{Points Value} / \text{Steals Value} \times \text{Steals Input}) + \\
 & (\text{Points Value} / \text{Blocks Value} \times \text{Blocks Input}) + \\
 & (\text{Points Value} / \text{Threes Value} \times \text{Threes Input}) + \\
 & (\text{Points Value} / \text{Field Goals Attempted Value} \times \text{Field Goals Attempted Input}) \times (((\text{Input} \\
 & \text{Field Goals Made} / \text{Input Field Goals Attempted}) - \text{Field Goal Percentage Value}) / \text{Field} \\
 & \text{Goal Percentage Value}) + \\
 & (\text{Points Value} / \text{Free Throws Attempted Value} \times \text{Free Throws Attempted Input}) \times (((\text{Input} \\
 & \text{Free Throws Made} / \text{Input Free Throws Attempted}) - \text{Free Throw Percentage Value}) / \\
 & \text{Free Throw Percentage Value}) - \\
 & (\text{Points Value} / \text{Turnovers Value} \times \text{Turnovers Input})
 \end{aligned}$$

Solution

- Can now compute the performance metric for the players from the problem statement

Performance Metric	Points	Rebounds	Assists	Steals	Blocks	Threes	Field Goal %	Free Throw %	Turnovers
64.68	23	3.9	5.2	1.1	.3	1.1	45.8%	82.9%	2.2
80.09	13.9	10.8	.9	.8	1.9	0	66%	54.3%	1.4

- It is now shown that the bottom player is considerably better, and will affect an individual's fantasy team more positively than the top player

Process: Data Scraping

- Eleven values are pulled from the NBA statistics leaderboards
- Implemented using the Requests library within Python
- Values are obtained by getting the page, and then the desired value at a specific location on the page
- Values are then written to a data.txt file to be used later in the application
- All work utilizing the Requests library was self-researched and implemented

Process: User Interface Version One

- First version was written in C++ and was a command-line based program
- Users would be prompted for statistics upon running
- Program would run the python web scraping file to obtain NBA data
- Would then read in the file that was created by the python script
- Program would take user input and compute statistic
- Would then output the computed statistic to the command line

Process: User Interface Final Version

- Final version is a Python-based Flask application
- Flask is a Python microframework that allows for the development of a web application
- Application has been developed using Python and HTML techniques
- Application runs the Python web scraping file, and uses the data from that file along with the user input data
- All work within the Flask application was self-researched and implemented

Process: Application Hosting

- A Flask application implementation allows for a working application that is accessible from the localhost of a computer
- The web service Heroku allows for an application to be deployable so that it is accessible from all computers that can access the website
- Application is currently deployed to Heroku as a running Flask application, and is a fully functioning and accessible website at:
<https://basketball-performance-metric.herokuapp.com/>

Testing

- Pulling data from the NBA leaderboards
 - Have verified that the page urls and xpath values are accurate and have been in use without any issues since early 2018
- Storing the pulled values in the necessary format
 - Have ensured that the file is opened and closed in the correct format for writing, and the values are written in the correct order
- Reading the data
 - Have ensured that the file is opened and closed in the correct format for reading, and the values are read in the correct order

Testing

- Taking input from the user
 - Flask fields allow for validators to be used to ensure that reasonable numbers are put into the input fields
- Calculating the metric using the pulled values and the user input
 - Have manually calculated the score for a select few players and verified that the application gave the correct result
- Outputting the metrics in the necessary format
 - Have ensured the store and sort functionality of the display process works as desired

Data Analysis

- Per the NBA statistical minimums, a player must play in 70% of their team's games to have their statistics qualify
- As an NBA team plays 82 games, players must play 58 games to qualify
- During the 2017-2018 NBA season, 259 players played 58 games
- Computation of the performance metric for each of the qualifying players gives the following data

Data Analysis

- Maximum Value: 124.06
 - 3rd Quartile Value: 60.40
 - Median Value: 46.30
 - 1st Quartile Value: 36.71
 - Minimum Value: 11.51
-
- Average Value: 50.07
 - Standard Deviation: 19.61

Qualified NBA Statistics 2017-2018

PM	Player	PTS	TRB	AST	STL	BLK	3P	FG	FGA	FT	FTA	TOV	ND
124.06	Anthony Davis	28.1	11.1	2.3	1.5	2.6	0.7	10.4	19.5	6.6	8	2.2	1.6536E-05
113.76	James Harden	30.4	5.4	8.8	1.8	0.7	3.7	9	20.1	8.7	10.1	4.4	0.00010443
108.72	Kevin Durant	26.4	6.8	5.4	0.7	1.8	2.5	9.3	18	5.3	5.9	3	0.00023273
104.07	LeBron James	27.5	8.6	9.1	1.4	0.9	1.8	10.5	19.3	4.7	6.5	4.2	0.00045974
100.26	Karl-Anthony Towns	21.3	12.3	2.4	0.8	1.4	1.5	7.8	14.3	4.2	4.9	1.9	0.00077014
99.35	Giannis Antetokounmpo	26.9	10	4.8	1.5	1.4	0.6	9.9	18.7	6.5	8.5	3	0.00086628
96.49	Victor Oladipo	23.1	5.2	4.3	2.4	0.8	2.1	8.5	17.9	3.9	4.9	2.9	0.00123635
96.23	Chris Paul	18.6	5.4	7.9	1.7	0.2	2.5	6.3	13.8	3.5	3.8	2.2	0.00127564
95.52	Andre Drummond	15	16	3	1.5	1.6	0	6	11.3	3.1	5.1	2.6	0.00138816
93.71	Nikola Jokic	18.5	10.7	6.1	1.2	0.8	1.5	6.7	13.5	3.5	4.2	2.8	0.00171118
91.95	Damian Lillard	26.9	4.5	6.6	1.1	0.4	3.1	8.5	19.4	6.8	7.4	2.8	0.00208165
91.24	Russell Westbrook	25.4	10.1	10.3	1.8	0.3	1.2	9.5	21.1	5.2	7.1	4.8	0.00224744
90.32	Paul George	21.9	5.7	3.3	2	0.5	3.1	7.3	17	4.3	5.2	2.7	0.00247724
89.18	Draymond Green	11	7.6	7.3	1.4	1.3	1.1	4	8.8	1.9	2.5	2.9	0.00278632
87.18	Jimmy Butler	22.2	5.3	4.9	2	0.4	1.2	7.4	15.6	6.2	7.2	1.8	0.0033968
84.78	Kyle Lowry	16.2	5.6	6.9	1.1	0.2	3.1	5.2	12.1	2.9	3.3	2.3	0.0042497
82.93	Joel Embiid	22.9	11	3.2	0.6	1.8	1	8.1	16.8	5.7	7.4	3.7	0.00499934
82.91	Robert Covington	12.6	5.4	2	1.7	0.9	2.5	4.3	10.5	1.5	1.7	1.6	0.00500788
82.59	Jrue Holiday	19	4.5	6	1.5	0.8	1.5	7.6	15.4	2.3	2.9	2.6	0.00514587
82.19	Kyrie Irving	24.4	3.8	5.1	1.1	0.3	2.8	8.9	18.1	3.9	4.4	2.3	0.00532173
80.52	Ben Simmons	15.8	8.1	8.2	1.7	0.9	0	6.7	12.3	2.4	4.2	3.4	0.00609578
80.09	Clint Capela	13.9	10.8	0.9	0.8	1.9	0	6	9.1	1.9	3.5	1.4	0.00630529
79.47	Marc Gasol	17.2	8.1	4.2	0.7	1.4	1.5	5.9	14.2	3.8	4.6	2.7	0.00661452
78.47	Kemba Walker	22.1	3.1	5.6	1.1	0.3	2.9	7.4	17	4.5	5.3	2.2	0.00713052
77.84	Otto Porter	14.7	6.4	2	1.5	0.5	1.8	5.8	11.5	1.4	1.7	1	0.00746611
77.56	Eric Bledsoe	17.7	3.8	5	2	0.6	1.7	6.3	13.4	3.4	4.3	2.9	0.00761776
75.9	Al Horford	12.9	7.4	4.7	0.6	1.1	1.3	5.1	10.5	1.3	1.7	1.8	0.00854642
74.56	Aaron Gordon	17.6	7.9	2.3	1	0.8	2	6.5	14.9	2.7	3.9	1.8	0.00932914
74.08	Bradley Beal	22.6	4.4	4.5	1.2	0.4	2.4	8.3	18.1	3.6	4.5	2.6	0.0096157
73.91	Kevin Love	17.6	9.3	1.7	0.7	0.4	2.3	5.7	12.4	4	4.5	1.5	0.0097179
73.81	Khrist Middleton	20.1	5.2	4	1.5	0.3	1.8	7.2	15.5	3.9	4.4	2.3	0.00977817
72.56	Klay Thompson	20	3.8	2.5	0.8	0.5	3.1	7.9	16.2	1.1	1.3	1.8	0.0105408
70.89	Myles Turner	12.7	6.4	1.3	0.6	1.8	0.9	4.7	9.8	2.5	3.2	1.5	0.01157966
70.87	Josh Richardson	12.9	3.5	2.9	1.5	0.9	1.6	4.9	10.9	1.5	1.8	1.7	0.01159219

Qualified NBA Statistics 2017-2018

PM	Player	PTS	TRB	AST	STL	BLK	3P	FG	FGA	FT	FTA	TOV	ND
70.61	Gary Harris	17.5	2.6	2.9	1.8	0.2	2.3	6.6	13.6	2	2.4	1.8	0.01175525
70.31	Will Barton	15.7	5	4.1	1	0.6	1.9	5.7	12.7	2.2	2.8	1.8	0.01194365
70.24	Tobias Harris	18.6	5.5	2.4	0.9	0.4	2.3	7	15.3	2.2	2.7	1.3	0.01198763
69.62	Blake Griffin	21.4	7.4	5.8	0.7	0.3	1.9	7.5	17.2	4.5	5.7	2.8	0.01237749
69.01	CJ McCollum	21.4	4	3.4	1	0.4	2.3	8.2	18.6	2.6	3.1	1.9	0.01276099
68.03	Donovan Mitchell	20.5	3.7	3.7	1.5	0.3	2.4	7.5	17.2	3	3.8	2.7	0.01337501
68.03	Trevor Ariza	11.7	4.4	1.6	1.5	0.2	2.5	4	9.7	1.1	1.3	0.8	0.01337501
66.95	Joe Ingles	11.5	4.2	4.8	1.1	0.2	2.5	4.1	8.8	0.8	1	1.9	0.0140453
66.67	Lauri Markkanen	15.2	7.5	1.2	0.6	0.6	2.1	5.5	12.7	2.1	2.4	1.2	0.01421746
66.6	Jeff Teague	14.2	3	7	1.5	0.3	1.2	5.1	11.3	2.9	3.4	2.5	0.01426037
65.96	LaMarcus Aldridge	23.1	8.5	2	0.6	1.2	0.4	9.2	18	4.5	5.3	1.5	0.01465013
65.72	DeAndre Jordan	12	15.2	1.5	0.5	0.9	0	4.8	7.5	2.4	4.1	1.8	0.01479496
65.43	Lou Williams	22.6	2.5	5.3	1.1	0.2	2.4	7.4	16.9	5.5	6.2	3	0.01496888
65.17	entavious Caldwell-Pope	13.4	5.2	2.2	1.4	0.2	2.1	4.6	10.8	2.1	2.6	1.3	0.01512373
65.17	Darren Collison	12.4	2.6	5.3	1.3	0.2	1.4	4.6	9.2	1.8	2.1	1.2	0.01512373
64.92	Dwight Howard	16.6	12.5	1.3	0.6	1.6	0	6.2	11.2	4.1	7.2	2.6	0.01527161
64.92	Kent Bazemore	12.9	3.8	3.5	1.5	0.7	1.7	4.4	10.4	2.5	3.1	2.4	0.01527161
64.68	DeMar DeRozan	23	3.9	5.2	1.1	0.3	1.1	8.1	17.7	5.8	7	2.2	0.01541258
64.59	Spencer Dinwiddie	12.6	3.2	6.6	0.9	0.3	1.8	4.1	10.5	2.7	3.4	1.6	0.01546518
63.5	Serge Ibaka	12.6	6.3	0.8	0.4	1.3	1.4	5	10.3	1.2	1.6	1.2	0.01608976
62.94	Jayson Tatum	13.9	5	1.6	1	0.7	1.3	5	10.4	2.7	3.2	1.4	0.01640068
62.81	Steven Adams	13.9	9	1.2	1.2	1	0	5.9	9.4	2.1	3.8	1.7	0.0164718
62.5	Thaddeus Young	11.8	6.3	1.9	1.7	0.4	0.7	5.2	10.7	0.7	1.1	1.3	0.01663967
62.33	Dirk Nowitzki	12	5.7	1.6	0.6	0.6	1.8	4.5	9.8	1.3	1.4	0.7	0.01673068
62.28	Kyle Anderson	7.9	5.4	2.7	1.6	0.8	0.3	3.1	5.9	1.4	2	1.3	0.0167573
62.11	Nicolas Batum	11.6	4.8	5.5	1	0.4	1.4	4.2	10.2	1.7	2	2	0.01684732
61.94	Wesley Matthews	12.7	3.1	2.7	1.2	0.3	2.4	4.5	11.1	1.3	1.6	1.3	0.01693655
61.48	Jusuf Nurkic	14.3	9	1.8	0.8	1.4	0	6.1	12	2.2	3.5	2.3	0.01717389
61.27	Allen Crabbe	13.2	4.3	1.6	0.6	0.5	2.7	4.5	11	1.5	1.8	1	0.01728019
61.1	Danny Green	8.6	3.6	1.6	0.9	1.1	1.7	3.1	8	0.7	0.9	1.1	0.01736526
60.4	DeMarre Carroll	13.5	6.6	2	0.8	0.4	2	4.5	10.8	2.6	3.4	1.4	0.01770598
60.39	Carmelo Anthony	16.2	5.8	1.3	0.6	0.6	2.2	6.1	15	1.9	2.5	1.3	0.01771073
60.26	Brook Lopez	13	4	1.7	0.4	1.3	1.5	5	10.7	1.5	2.1	1.3	0.01777221
60.02	Austin Rivers	15.1	2.4	4	1.2	0.3	2.2	5.6	13.2	1.7	2.6	1.8	0.01788419

Qualified NBA Statistics 2017-2018

PM	Player	PTS	TRB	AST	STL	BLK	3P	FG	FGA	FT	FTA	TOV	ND
59.91	Jamal Murray	16.7	3.7	3.4	1	0.3	2	5.9	13.1	2.8	3.1	2.1	0.01793486
59.49	Dario Saric	14.6	6.7	2.6	0.7	0.3	2	5.2	11.4	2.3	2.7	1.9	0.01812439
59.22	Ricky Rubio	13.1	4.6	5.3	1.6	0.1	1.2	4.5	10.7	2.9	3.3	2.7	0.01824286
59.19	Justin Holiday	12.2	4	2.1	1.1	0.4	2.2	4	10.9	1.9	2.3	1.3	0.01825586
58.87	Taurean Waller-Prince	14.1	4.7	2.6	1	0.5	2.1	5.1	11.9	1.9	2.2	2.3	0.01839241
58.6	Pau Gasol	10.1	8	3.1	0.3	1	0.6	3.7	8.1	2.1	2.7	1.4	0.01850459
58.59	Rajon Rondo	8.3	4	8.2	1.1	0.2	0.8	3.6	7.6	0.4	0.7	2.3	0.01850869
58.43	Eric Gordon	18	2.5	2.2	0.6	0.4	3.2	6	14.1	2.8	3.5	1.9	0.01857376
58.2	Terry Rozier	11.3	4.7	2.9	1	0.2	1.9	4	10	1.4	1.9	1	0.01866553
57.87	Willie Cauley-Stein	12.8	7	2.4	1.1	0.9	0	5.3	10.6	2.1	3.4	1.5	0.01879348
57.63	Buddy Hield	13.5	3.8	1.9	1.1	0.3	2.2	5.2	11.7	0.9	1	1.6	0.01888372
57.29	Al-Farouq Aminu	9.3	7.6	1.2	1.1	0.6	1.8	3.3	8.4	0.9	1.2	1.1	0.01900744
56.99	John Henson	8.8	6.8	1.5	0.6	1.4	0	3.8	6.6	1.2	2.1	1.1	0.01911125
56.8	Andrew Wiggins	17.7	4.4	2	1.1	0.6	1.4	6.9	15.9	2.5	3.8	1.7	0.01917703
56.59	James Johnson	10.8	4.9	3.8	1	0.7	0.8	4.2	8.4	1.5	2.2	1.9	0.01924649
56.47	Kyle Kuzma	16.1	6.3	1.8	0.6	0.4	2.1	6.1	13.5	1.9	2.7	1.8	0.01928531
56.27	Dewayne Dedmon	10	7.9	1.5	0.6	0.8	0.8	4	7.7	1.1	1.4	1.4	0.01934856
56.24	J.J. Redick	17.1	2.5	3	0.5	0.1	2.8	5.8	12.6	2.8	3.1	1.5	0.01935789
55.67	Goran Dragic	17.3	4.1	4.8	0.8	0.2	1.5	6.5	14.3	2.9	3.6	2.2	0.01952739
55.65	Dennis Schroder	19.4	3.1	6.2	1.1	0.1	1.1	7.5	17.1	3.4	4	2.7	0.01953306
55.15	Rondae Hollis-Jefferson	13.9	6.8	2.5	1	0.7	0.2	5	10.6	3.7	4.6	1.8	0.01966882
55.12	Jonas Valanciunas	12.7	8.6	1.1	0.4	0.9	0.4	5.1	8.9	2.2	2.7	1.5	0.01967659
54.72	Enes Kanter	14.1	11	1.5	0.5	0.5	0	5.9	10	2.2	2.6	1.7	0.01977604
54.71	Kelly Olynyk	11.5	5.7	2.7	0.8	0.5	1.4	4.2	8.4	1.8	2.3	1.9	0.01977843
54.64	Tyler Johnson	11.7	3.4	2.3	0.8	0.5	1.7	4.3	9.9	1.5	1.8	1.1	0.019795
54.45	John Collins	10.5	7.3	1.3	0.6	1.1	0.2	4.2	7.4	1.8	2.5	1.4	0.01983879
54.41	Larry Nance	8.7	6.8	1.2	1.3	0.6	0	3.6	6.3	1.4	2.1	0.8	0.01984778
54.18	T.J. Warren	19.6	5.1	1.3	1	0.6	0.3	8.1	16.4	3	3.9	1.3	0.01989797
54.15	Kyle O'Quinn	7.1	6.1	2.1	0.5	1.3	0.1	2.9	5	1.3	1.6	1.3	0.01990432
53.97	Derrick Favors	12.3	7.2	1.3	0.7	1.1	0.2	5.1	9.1	1.8	2.8	1.1	0.0199415
53.76	Harrison Barnes	18.9	6.1	2	0.6	0.2	1.5	7	15.7	3.4	4.1	1.5	0.01998285
53.52	Denzel Valentine	10.2	5.1	3.2	0.8	0.1	1.9	3.9	9.4	0.5	0.7	1.3	0.02002739
53.45	Elfrid Payton	12.7	4.3	6.2	1.3	0.3	0.5	5.2	10.6	1.7	2.7	2.7	0.02003983
53.31	Ersan Ilyasova	10.9	5.9	1.3	0.9	0.4	1.3	4	8.9	1.5	2	1	0.02006398

Qualified NBA Statistics 2017-2018

PM	Player	PTS	TRB	AST	STL	BLK	3P	FG	FGA	FT	FTA	TOV	ND
52.98	Taj Gibson	12.2	7.1	1.2	0.8	0.7	0.1	5.2	9	1.7	2.3	1.1	0.02011695
52.59	Kelly Oubre	11.8	4.5	1.2	1	0.4	1.6	3.9	9.8	2.4	2.9	1.1	0.02017238
52.5	Courtney Lee	12	2.9	2.4	1.1	0.2	1.5	4.5	9.9	1.5	1.6	1.1	0.02018405
52.36	Marvin Williams	9.5	4.7	1.2	0.7	0.5	1.6	3.3	7.2	1.3	1.6	0.8	0.02020139
52.09	Brandon Ingram	16.1	5.3	3.9	0.8	0.7	0.7	6.1	12.9	3.3	4.8	2.5	0.02023195
51.89	Wayne Ellington	11.2	2.8	1	0.7	0.1	2.9	3.7	9.1	0.9	1	0.7	0.02025215
51.88	Jaylen Brown	14.5	4.9	1.6	1	0.4	1.7	5.3	11.5	2.1	3.3	1.8	0.0202531
51.57	Caris LeVert	12.1	3.7	4.2	1.2	0.3	1.2	4.5	10.4	1.9	2.6	2.2	0.02028011
51.53	Jeremy Lamb	12.9	4.1	2.3	0.8	0.4	1.2	4.8	10.4	2.2	2.6	1.2	0.02028323
51.12	Dennis Smith	15.2	3.8	5.2	1	0.3	1.5	5.9	14.8	1.9	2.8	2.8	0.02031036
50.4	Bobby Portis	13.2	6.8	1.7	0.7	0.3	1.1	5.2	11.1	1.6	2.1	1.4	0.02033656
50.22	Rodney Hood	14.7	2.8	1.6	0.8	0.2	2.1	5.4	12.5	1.9	2.2	1.2	0.02033884
50.12	Bogdan Bogdanovic	11.8	2.9	3.3	0.9	0.2	1.7	4.4	9.9	1.3	1.5	1.6	0.02033936
49.93	Wilson Chandler	10	5.4	2.1	0.6	0.5	1.2	3.8	8.5	1.2	1.5	1.2	0.0203389
49.91	E'Twaun Moore	12.5	2.9	2.3	1	0.1	1.6	5.2	10.1	0.6	0.8	1.2	0.02033874
49.24	Reggie Bullock	11.3	2.5	1.5	0.8	0.2	2	4.3	8.8	0.6	0.8	0.8	0.02032118
49.03	Markieff Morris	11.5	5.6	1.9	0.8	0.5	1	4.5	9.4	1.4	1.8	1.7	0.0203108
48.97	Fred VanVleet	8.6	2.4	3.2	0.9	0.3	1.4	3.1	7.2	1.1	1.3	1	0.02030741
48.32	Mario Hezonja	9.6	3.7	1.4	1.1	0.4	1.2	3.6	8.2	1.1	1.4	1.1	0.02025854
48.23	Delon Wright	8	2.9	2.9	1	0.5	0.8	2.9	6.3	1.4	1.7	1.1	0.02025003
47.92	Ryan Anderson	9.3	5	0.9	0.4	0.3	2	3.1	7.3	1.1	1.4	0.6	0.02021749
47.85	George Hill	10	2.7	2.8	0.9	0.4	1.3	3.5	7.7	1.6	2.1	1.3	0.02020945
47.8	J.J. Barea	11.6	2.9	6.3	0.5	0	1.7	4.4	10	1.2	1.5	2.1	0.02020355
47.72	P.J. Tucker	6.1	5.6	0.9	1	0.3	1.4	2.1	5.4	0.5	0.7	0.9	0.02019385
46.96	Marquese Chriss	7.7	5.5	1.2	0.7	1	0.8	2.8	6.6	1.3	2.2	1.5	0.02008519
46.63	Bojan Bogdanovic	14.3	3.4	1.5	0.7	0.1	1.9	5.1	10.7	2.2	2.6	1.3	0.02002883
46.3	Jarrett Allen	8.2	5.4	0.7	0.4	1.2	0.1	3.3	5.5	1.6	2	1.1	0.01996698
46.3	Andre Iguodala	6	3.8	3.3	0.8	0.6	0.5	2.3	5	0.9	1.4	1	0.01996698
46.26	Justise Winslow	7.8	5.4	2.2	0.8	0.5	0.7	3	7.2	1	1.5	1.1	0.01995911
46.2	Marco Belinelli	12.1	1.9	1.9	0.8	0.2	1.9	4.2	9.6	1.9	2	1.2	0.01994716
45.64	Zach Randolph	14.5	6.7	2.2	0.7	0.2	0.9	6.1	12.9	1.4	1.8	2	0.019827
45.62	Dwyane Wade	11.4	3.8	3.4	0.9	0.7	0.5	4.5	10.2	2	2.8	2.1	0.01982242
45.42	Stanley Johnson	8.7	3.7	1.6	1.4	0.2	1	3	8.1	1.6	2.1	1.1	0.01977558
45.37	Alex Len	8.5	7.5	1.2	0.4	0.9	0	3.1	5.5	2.3	3.3	1.1	0.01976356

Qualified NBA Statistics 2017-2018

PM	Player	PTS	TRB	AST	STL	BLK	3P	FG	FGA	FT	FTA	TOV	ND
45.3	Mason Plumlee	7.1	5.4	1.9	0.7	1.1	0	3	5	1.1	2.4	1.4	0.01974654
45.22	Wesley Johnson	5.4	2.9	0.8	1	0.8	0.8	2	5	0.6	0.8	0.7	0.0197268
45.17	Yogi Ferrell	10.2	3	2.5	0.8	0.1	1.6	3.8	8.9	1	1.3	1	0.0197143
45.15	Kyle Korver	9.2	2.3	1.2	0.4	0.4	2.2	3	6.6	0.9	1	0.8	0.01970927
45.11	Dejounte Murray	8.1	5.7	2.9	1.2	0.4	0.1	3.3	7.5	1.3	1.8	1.7	0.01969914
44.97	Dwight Powell	8.5	5.6	1.2	0.8	0.4	0.4	3.2	5.4	1.7	2.3	0.7	0.01966311
44.84	Marcin Gortat	8.4	7.6	1.8	0.5	0.7	0	3.5	6.8	1.3	2	1.2	0.01962882
44.27	Jakob Poeltl	6.9	4.8	0.7	0.5	1.2	0	3.1	4.7	0.7	1.2	1	0.01946905
44.06	Pascal Siakam	7.3	4.5	2	0.8	0.5	0.4	3.1	6.1	0.7	1.1	0.8	0.01940639
43.87	C.J. Miles	10	2.2	0.8	0.5	0.3	2.3	3.2	8.6	1.2	1.4	0.6	0.01934796
43.85	Trey Lyles	9.9	4.8	1.2	0.4	0.5	1.1	3.7	7.5	1.4	2	0.8	0.01934171
43.82	Cory Joseph	7.9	3.2	3.2	1	0.2	0.8	3.1	7.4	0.9	1.1	1.1	0.01933231
43.81	Joe Harris	10.8	3.3	1.6	0.4	0.3	1.9	3.9	8	1.1	1.3	1.2	0.01932917
43.47	J.R. Smith	8.3	2.9	1.8	0.9	0.1	1.8	3	7.4	0.5	0.7	1	0.01921962
43.17	Garrett Temple	8.4	2.3	1.9	0.9	0.4	1.4	3.1	7.3	0.9	1.2	1.2	0.01911871
43.13	Anthony Tolliver	8.9	3.1	1.1	0.4	0.3	2	2.8	6	1.3	1.7	0.7	0.01910496
43.08	T.J. McConnell	6.3	3	4	1.2	0.2	0.4	2.8	5.5	0.4	0.5	1.5	0.01908767
43.02	Julius Randle	16.1	8	2.6	0.5	0.5	0.1	6.1	11	3.7	5.2	2.6	0.01906678
42.95	Jae Crowder	9.7	3.4	1.2	0.8	0.3	1.4	3.4	8.4	1.5	1.9	0.9	0.01904221
42.87	Patty Mills	10	1.9	2.8	0.7	0.1	1.9	3.4	8.3	1.3	1.4	1.3	0.01901388
42.79	Josh Hart	7.9	4.2	1.3	0.7	0.3	1.2	2.8	6	1	1.5	0.7	0.01898527
42.48	Ish Smith	10.9	2.7	4.4	0.8	0.2	0.4	4.7	9.8	1	1.4	1.3	0.01887185
42.41	Michael Beasley	13.2	5.6	1.7	0.5	0.6	0.5	5.5	10.9	1.7	2.1	1.9	0.01884569
42.39	Jerian Grant	8.4	2.3	4.6	0.9	0.1	0.8	2.8	6.8	1.9	2.5	1.2	0.01883817
42.32	Luc Mbah a Moute	7.5	3	0.9	1.2	0.4	1	2.7	5.7	1.1	1.6	1.1	0.01881174
42.32	Maurice Harkless	6.5	2.7	0.9	0.8	0.7	0.8	2.5	5	0.7	1	0.7	0.01881174
42.04	Kosta Koufos	6.7	6.6	1.2	0.7	0.5	0	3.1	5.5	0.5	1	0.7	0.01870402
41.91	David West	6.8	3.3	1.9	0.6	1	0	3	5.2	0.8	1.1	1.1	0.01865292
41.64	Ekpe Udoh	2.6	2.4	0.8	0.7	1.2	0	1	1.9	0.7	0.9	0.3	0.01854463
41.46	Jordan Clarkson	13.9	2.7	2.7	0.7	0.1	1.4	5.3	11.8	1.9	2.3	1.6	0.01847085
41.35	Jerami Grant	8.4	3.9	0.7	0.4	1	0.4	3	5.6	2	3	0.7	0.01842513
41.31	Tomas Satoransky	7.2	3.2	3.9	0.7	0.2	0.6	2.8	5.3	1	1.3	1.1	0.0184084
40.96	D.J. Augustin	10.2	2.1	3.8	0.7	0	1.5	3.3	7.2	2.2	2.5	1.6	0.01825935
40.91	Shabazz Napier	8.7	2.3	2	1.1	0.2	1.1	3	7.2	1.6	1.9	1.2	0.01823768

Qualified NBA Statistics 2017-2018

PM	Player	PTS	TRB	AST	STL	BLK	3P	FG	FGA	FT	FTA	TOV	ND
40.87	Domantas Sabonis	11.6	7.7	2	0.5	0.4	0.2	4.6	8.9	2.3	3	1.9	0.01822028
40.77	Bismack Biyombo	5.7	5.7	0.8	0.3	1.2	0	2.2	4.3	1.2	1.9	1	0.01817652
40.37	Frank Kaminsky	11.1	3.6	1.6	0.5	0.2	1.3	3.9	9.1	2	2.5	0.8	0.01799785
40.35	Josh Jackson	13.1	4.6	1.5	1	0.5	0.7	5.1	12.1	2.3	3.6	1.9	0.01798877
40.14	Tony Snell	6.9	1.9	1.3	0.6	0.4	1.4	2.5	5.7	0.5	0.6	0.5	0.01789254
39.47	Dragan Bender	6.5	4.4	1.6	0.3	0.6	1.4	2.3	5.9	0.5	0.6	1.4	0.01757549
39.04	Nemanja Bjelica	6.8	4.1	1.3	0.7	0.2	1.1	2.6	5.5	0.5	0.7	0.8	0.0173643
38.92	De'Aaron Fox	11.6	2.8	4.4	1	0.3	0.6	4.5	10.9	1.9	2.7	2.4	0.01730434
38.92	Ed Davis	5.3	7.4	0.5	0.4	0.7	0	2.2	3.7	0.9	1.4	0.8	0.01730434
38.58	Lance Stephenson	9.2	5.2	2.9	0.6	0.2	0.8	3.7	8.6	1	1.5	1.6	0.01713207
38.45	Jonathon Simmons	13.9	3.5	2.5	0.8	0.2	1	5.1	11	2.7	3.5	2.1	0.0170653
38.43	Skal Labissiere	8.7	4.8	1.2	0.4	0.8	0.3	3.4	7.6	1.6	2	1.2	0.01705498
38.19	Tyus Jones	5.1	1.6	2.8	1.2	0.1	0.5	1.8	4	0.9	1	0.7	0.01693031
38.09	Mario Chalmers	7.7	2.4	3	1.2	0.2	0.8	2.5	6.7	1.8	2.1	1.8	0.01687789
37.85	Montrezl Harrell	11	4	1	0.5	0.7	0	4.6	7.2	1.8	2.9	0.9	0.01675096
37.72	Dillon Brooks	11	3.1	1.6	0.9	0.2	1.1	4.1	9.4	1.5	2	1.5	0.01668156
37.7	Daniel Theis	5.3	4.3	0.9	0.5	0.8	0.3	2	3.7	1	1.3	0.9	0.01667084
37.34	Quincy Acy	5.9	3.7	0.8	0.5	0.4	1.5	1.9	5.2	0.7	0.9	0.9	0.0164762
37.24	Amir Johnson	4.6	4.5	1.6	0.6	0.6	0.1	1.9	3.5	0.7	1.1	0.7	0.01642155
37.22	Vince Carter	5.4	2.6	1.2	0.7	0.4	1	2	4.9	0.5	0.6	0.6	0.01641059
36.96	Jeff Green	10.8	3.2	1.3	0.5	0.4	0.7	3.8	7.9	2.6	3	1	0.01626726
36.95	Maxi Kleber	5.4	3.3	0.7	0.4	0.7	0.6	2.1	4.3	0.6	0.8	0.4	0.01626171
36.95	Salah Mejri	3.5	4	0.6	0.4	1.1	0	1.4	2.2	0.6	1.1	0.6	0.01626171
36.71	Jarell Martin	7.7	4.4	1	0.5	0.7	0.5	3	6.8	1.2	1.6	1.2	0.01612794
36.71	Robin Lopez	11.8	4.5	1.9	0.2	0.8	0.1	5.3	10.1	1.1	1.4	1.7	0.01612794
36.58	Bam Adebayo	6.9	5.5	1.5	0.5	0.6	0	2.5	4.9	1.9	2.6	1	0.01605494
36.39	David Nwaba	7.9	4.7	1.5	0.8	0.4	0.3	2.9	6	1.9	2.9	1.1	0.01594757
36.14	Shelvin Mack	6.9	2.4	3.9	0.8	0.1	0.7	2.6	6	1	1.4	1.3	0.01580514
35.73	Darius Miller	7.8	2	1.4	0.3	0.2	1.8	2.6	5.9	0.7	0.8	0.7	0.01556882
35.7	Jarrett Jack	7.5	3.1	5.6	0.6	0.1	0.4	3	7.1	1	1.2	1.9	0.0155514
35.42	Tyler Ulis	7.8	1.8	4.4	1	0.1	0.6	3	7.8	1.2	1.4	1.8	0.01538803
35.18	Dante Cunningham	5.7	4.1	0.7	0.5	0.4	0.8	2.2	5	0.4	0.7	0.5	0.01524688
35.18	Gorgui Dieng	5.9	4.6	0.9	0.6	0.5	0.2	2.4	4.9	1	1.3	0.7	0.01524688
34.69	Luke Kennard	7.6	2.4	1.7	0.6	0.2	1.1	2.8	6.4	0.9	1	0.9	0.01495577

Qualified NBA Statistics 2017-2018

PM	Player	PTS	TRB	AST	STL	BLK	3P	FG	FGA	FT	FTA	TOV	ND
34.61	Eric Moreland	2.1	4.1	1.2	0.5	0.8	0	1	1.8	0.2	0.4	0.6	0.01490789
34.56	Evan Turner	8.2	3.1	2.2	0.6	0.4	0.5	3.3	7.3	1.2	1.4	1.3	0.01487791
34.46	Devin Harris	8.4	1.8	2.1	0.7	0.2	1.1	2.6	6.4	2	2.4	1.1	0.01481786
34.27	Michael Kidd-Gilchrist	9.2	4.1	1	0.7	0.4	0	3.8	7.6	1.6	2.3	0.7	0.01470336
34.08	Troy Daniels	8.9	1.6	0.6	0.3	0.1	2.3	2.9	7.3	0.7	0.8	0.7	0.01458839
34.06	James Ennis	7.1	3.1	1	0.7	0.3	0.7	2.5	5.3	1.3	1.6	0.8	0.01457626
33.96	Raymond Felton	6.9	1.9	2.5	0.6	0.2	1	2.7	6.7	0.4	0.5	0.9	0.01451553
33.59	Manu Ginobili	8.9	2.2	2.5	0.7	0.2	1	3.1	7.1	1.7	2	1.6	0.01428981
32.32	Davis Bertans	5.9	2	1	0.3	0.4	1.2	2.1	4.8	0.5	0.6	0.5	0.0135048
32.11	Thon Maker	4.8	3	0.6	0.5	0.7	0.4	1.8	4.3	0.9	1.3	0.6	0.01337381
32.04	Jamal Crawford	10.3	1.2	2.3	0.5	0.1	1.3	3.9	9.3	1.3	1.4	1.2	0.01333009
31.91	JaVale McGee	4.8	2.6	0.5	0.3	0.9	0	2.1	3.4	0.6	0.8	0.4	0.01324883
31.81	OG Anunoby	5.9	2.5	0.7	0.7	0.2	1	2.2	4.7	0.5	0.8	0.6	0.01318626
31.77	Kevon Looney	4	3.3	0.6	0.5	0.8	0	1.7	2.9	0.6	1.2	0.5	0.01316122
31.72	Langston Galloway	6.2	1.6	1	0.6	0.1	1.3	2.2	5.9	0.6	0.7	0.3	0.01312991
30.65	Corey Brewer	5.3	2.1	0.9	1.1	0.2	0.4	2	4.4	0.9	1.3	0.7	0.01245804
30.2	Nick Young	7.3	1.6	0.5	0.5	0.1	1.5	2.5	6.1	0.7	0.8	0.5	0.01217502
30.16	Patrick Patterson	3.9	2.4	0.7	0.6	0.3	0.8	1.3	3.2	0.5	0.6	0.4	0.01214987
30.03	Tyler Zeller	6.7	4.6	0.7	0.2	0.5	0.2	2.8	5.1	0.9	1.2	0.7	0.01206813
29.65	Zaza Pachulia	5.4	4.7	1.6	0.6	0.2	0	2.2	3.8	1.1	1.3	1	0.01182937
29.42	Aron Baynes	6	5.4	1.1	0.3	0.6	0	2.6	5.5	0.7	1	1	0.01168503
29.1	Frank Ntilikina	5.9	2.3	3.2	0.8	0.2	0.6	2.3	6.4	0.6	0.9	1.7	0.0114845
29.05	Deyonta Davis	5.8	4	0.6	0.2	0.6	0	2.6	4.3	0.6	0.9	0.5	0.0114532
28.98	Emmanuel Mudiay	8.6	2.4	3.2	0.6	0.2	0.7	3.2	8.2	1.5	2	1.8	0.01140941
28.95	Justin Jackson	6.7	2.8	1.1	0.4	0.2	0.8	2.7	6.2	0.4	0.5	0.6	0.01139065
28.68	Tim Frazier	3	1.9	3.3	0.8	0.1	0.3	1.2	2.9	0.4	0.5	1	0.01122198
28.58	Jonas Jerebko	5.8	3.3	0.6	0.3	0.2	0.9	2.1	4.6	0.6	0.8	0.4	0.01115962
28.42	Royce O'Neale	5	3.4	1.4	0.5	0.2	0.6	1.7	4.1	0.9	1.1	0.8	0.01105995
28.2	Alec Burks	7.7	3	1	0.6	0.1	0.7	2.7	6.5	1.7	1.9	0.9	0.01092317
28.09	Trevor Booker	6.3	4.7	1.2	0.4	0.3	0.1	2.6	5	1	1.4	0.9	0.01085491
27.99	Pat Connaughton	5.4	2	1.1	0.3	0.3	0.9	2	4.8	0.5	0.5	0.5	0.01079293
27.98	Doug McDermott	7.8	2.5	1	0.2	0.2	1.1	2.9	6.3	0.8	1	0.7	0.01078673
27.71	Mike Scott	8.8	3.3	1.1	0.3	0.1	0.9	3.6	6.9	0.7	1	1	0.01061978
25.48	Zach Collins	4.4	3.3	0.8	0.3	0.5	0.5	1.7	4.4	0.4	0.6	0.9	0.00926868

Qualified NBA Statistics 2017-2018

PM	Player	PTS	TRB	AST	STL	BLK	3P	FG	FGA	FT	FTA	TOV	ND
25.2	Alex Abrines	4.7	1.5	0.4	0.5	0.1	1.1	1.5	3.9	0.5	0.6	0.3	0.00910334
24.81	Ian Clark	7.4	1.7	1.5	0.4	0.1	0.8	3	6.7	0.6	0.8	0.8	0.00887494
24.58	Norman Powell	5.5	1.7	1.3	0.5	0.2	0.8	2.1	5.3	0.5	0.6	0.9	0.00874132
24.41	Ian Mahinmi	4.8	4.1	0.7	0.5	0.5	0	1.8	3.2	1.2	1.7	1.3	0.00864308
24.38	Shaun Livingston	5.5	1.8	2	0.5	0.3	0	2.4	4.8	0.7	0.9	0.8	0.0086258
24.2	Jodie Meeks	6.3	1.6	0.9	0.4	0.1	0.9	2	5.1	1.3	1.5	0.5	0.00852237
23.83	Sindarius Thornwell	3.9	1.9	0.9	0.7	0.3	0.3	1.4	3.2	0.8	1.2	0.7	0.00831146
23.76	Bryn Forbes	6.9	1.4	1	0.4	0	1.1	2.6	6.2	0.6	0.9	0.5	0.00827182
23.54	Malik Monk	6.7	1	1.4	0.3	0.1	1.3	2.4	6.7	0.5	0.6	0.8	0.00814778
23.33	Lance Thomas	4.1	2.4	0.6	0.4	0.2	0.7	1.4	3.7	0.6	0.7	0.5	0.00803018
22.94	Josh Huestis	2.3	2.3	0.3	0.2	0.6	0.5	0.9	2.6	0.1	0.3	0.4	0.00781387
21.58	Isaiah Taylor	6.6	1.4	3.1	0.5	0.1	0.3	2.4	5.8	1.5	2.1	1.3	0.00708222
21.22	Treveon Graham	4.3	1.9	0.9	0.5	0	0.6	1.5	3.5	0.7	0.9	0.4	0.00689474
20.05	DeAndre Liggins	1.7	1.3	0.8	0.6	0.2	0.3	0.7	1.8	0	0.1	0.3	0.00630434
19.53	Wesley Iwundu	3.7	2.2	0.9	0.5	0.2	0.1	1.5	3.5	0.5	0.8	0.5	0.00605152
19.19	Terrance Ferguson	3.1	0.8	0.3	0.4	0.2	0.7	1.1	2.8	0.1	0.2	0.2	0.00588948
16.89	Cedi Osman	3.9	2	0.7	0.4	0	0.5	1.5	3	0.4	0.8	0.5	0.00486309
15.89	Semi Ojeleye	2.7	2.2	0.3	0.3	0.1	0.5	0.9	2.6	0.3	0.6	0.3	0.00445545
15.01	Sam Dekker	4.2	2.4	0.5	0.3	0.1	0.1	1.8	3.5	0.5	0.8	0.5	0.00411622
13.26	Malik Beasley	3.2	1.1	0.5	0.2	0.1	0.5	1.2	2.9	0.4	0.5	0.4	0.00349548
11.51	Brandon Paul	2.3	1.1	0.6	0.4	0.1	0.2	0.9	2	0.3	0.7	0.5	0.00294481
50.071	Average												
19.6142	Standard Deviation												
11.51	Minimum Value												
36.71	1st Quartile												
46.3	Median Value												
60.395	3rd Quartile												
124.06	Maximum Value												

Requirements.txt

```
certifi==2018.10.15
chardet==3.0.4
Click==7.0
dominate==2.3.4
Flask==1.0.2
Flask-Bootstrap==3.3.7.1
Flask-Images==3.0.1
Flask-WTF==0.14.2
gunicorn==19.9.0
idna==2.7
itsdangerous==0.24
Jinja2==2.10
lxml==4.2.5
MarkupSafe==1.0
python-dotenv==0.9.1
requests==2.20.0
six==1.11.0
urllib3==1.24
visitor==0.1.3
Werkzeug==0.14.1
WTForms==2.2.1
```

data.py

```
from lxml import html
import requests

page0 = requests.get('http://www.espn.com/nba/statistics/player/_/stat/scoring-per-game')
tree0 = html.fromstring(page0.content)

pts = tree0.xpath('//*[@id="my-players-table"]/div/div[2]/table/tr[22]/td[6]/text()
[normalize-space()]')
pts = ' '.join(pts)

page1 = requests.get('http://www.espn.com/nba/statistics/player/_/stat/rebounds')
tree1 = html.fromstring(page1.content)

trb = tree1.xpath('//*[@id="my-players-table"]/div/div[2]/table/tr[22]/td[11]/text()
[normalize-space()]')
trb = ' '.join(trb)

page2 = requests.get('http://www.espn.com/nba/statistics/player/_/stat/assists')
tree2 = html.fromstring(page2.content)

ast = tree2.xpath('//*[@id="my-players-table"]/div/div[2]/table/tr[22]/td[7]/text()
[normalize-space()]')
ast = ' '.join(ast)

page3 = requests.get('http://www.espn.com/nba/statistics/player/_/stat/steals')
tree3 = html.fromstring(page3.content)

stl = tree3.xpath('//*[@id="my-players-table"]/div/div[2]/table/tr[22]/td[7]/text()
[normalize-space()]')
stl = ' '.join(stl)

page4 = requests.get('http://www.espn.com/nba/statistics/player/_/stat/blocks')
tree4 = html.fromstring(page4.content)

blk = tree4.xpath('//*[@id="my-players-table"]/div/div[2]/table/tr[22]/td[8]/text()
[normalize-space()]')
blk = ' '.join(blk)

page5 = requests.get('http://www.espn.com/nba/statistics/player/_/stat/scoring-per-
game/sort/fieldGoalPct')
tree5 = html.fromstring(page5.content)

fgp = tree5.xpath('//*[@id="my-players-table"]/div/div[2]/table/tr[22]/td[8]/text()
[normalize-space()]')
fgp = ' '.join(fgp)

page6 = requests.get('http://www.espn.com/nba/statistics/player/_/stat/scoring-per-
game/sort/freeThrowPct')
tree6 = html.fromstring(page6.content)

ftp = tree6.xpath('//*[@id="my-players-table"]/div/div[2]/table/tr[22]/td[12]/text()
[normalize-space()]')
ftp = ' '.join(ftp)

page7 = requests.get
('http://www.espn.com/nba/statistics/player/_/stat/turnovers/sort/avgTurnovers/order/fals
e')
```

~~data.txt~~

data.py

```
tree7 = html.fromstring(page7.content)
```

```
tov = tree7.xpath('//*[@id="my-players-table"]/div/div[2]/table/tr[22]/td[7]/text()  
[normalize-space()]')  
tov = ' '.join(tov)
```

```
page8 = requests.get  
('http://www.espn.com/nba/statistics/player/_/stat/3-points/sort/avgThreePointFieldGoalsM  
ade')  
tree8 = html.fromstring(page8.content)
```

```
tpm = tree8.xpath('//*[@id="my-players-table"]/div/div[2]/table/tr[24]/td[6]/text()  
[normalize-space()]')  
tpm = ' '.join(tpm)
```

```
page9 = requests.get('http://www.espn.com/nba/statistics/player/_/stat/field-  
goals/sort/avgFieldGoalsAttempted')  
tree9 = html.fromstring(page9.content)
```

```
fga = tree9.xpath('//*[@id="my-players-table"]/div/div[2]/table/tr[24]/td[7]/text()  
[normalize-space()]')  
fga = ' '.join(fga)
```

```
page10 = requests.get('http://www.espn.com/nba/statistics/player/_/stat/free-  
throws/sort/avgFreeThrowsAttempted')  
tree10 = html.fromstring(page10.content)
```

```
fta = tree10.xpath('//*[@id="my-players-table"]/div/div[2]/table/tr[24]/td[7]/text()  
[normalize-space()]')  
fta = ' '.join(fta)
```

```
fileout = open( "data.txt", 'w' )  
fileout.write( pts )  
fileout.write( "\n" )  
fileout.write( trb )  
fileout.write( "\n" )  
fileout.write( ast )  
fileout.write( "\n" )  
fileout.write( stl )  
fileout.write( "\n" )  
fileout.write( blk )  
fileout.write( "\n" )  
fileout.write( fgp )  
fileout.write( "\n" )  
fileout.write( ftp )  
fileout.write( "\n" )  
fileout.write( tov )  
fileout.write( "\n" )  
fileout.write( tpm )  
fileout.write( "\n" )  
fileout.write( fga )  
fileout.write( "\n" )  
fileout.write( fta )  
fileout.write( "\n" )  
fileout.close()
```

data.txt

21.3
9.6
6.0
1.50
1.42
.542
.900
3.1
2.5
17.5
5.9

app / --init -- .py

```
from flask import Flask
from config import Config
from flask_bootstrap import Bootstrap
from flask_images import Images
```

```
app = Flask(__name__)
app.config.from_object(Config)
```

```
from app import routes
bootstrap = Bootstrap(app)
images = Images(app)
```

app / forms.py

```
from flask_wtf import FlaskForm
from wtforms import StringField, DecimalField, HiddenField, SubmitField
from wtforms.validators import InputRequired, NumberRange
from decimal import Decimal
import data

class IndexForm(FlaskForm):
    name = StringField('Name', validators=[InputRequired()])
    points = DecimalField('Points', validators=[InputRequired(), NumberRange(0, 100,
    "Please enter a reasonable number")])
    rebounds = DecimalField('Rebounds', validators=[InputRequired(), NumberRange(0, 100,
    "Please enter a reasonable number")])
    assists = DecimalField('Assists', validators=[InputRequired(), NumberRange(0, 100,
    "Please enter a reasonable number")])
    steals = DecimalField('Steals', validators=[InputRequired(), NumberRange(0, 100,
    "Please enter a reasonable number")])
    blocks = DecimalField('Blocks', validators=[InputRequired(), NumberRange(0, 100,
    "Please enter a reasonable number")])
    threes = DecimalField('Three Pointers Made', validators=[InputRequired(), NumberRange
    (0, 100, "Please enter a reasonable number")])
    fgm = DecimalField('Field Goals Made', validators=[InputRequired(), NumberRange(0,
    100, "Please enter a reasonable number")])
    fga = DecimalField('Field Goals Attempted', validators=[InputRequired(), NumberRange
    (0, 100, "Please enter a reasonable number")])
    ftm = DecimalField('Free Throws Made', validators=[InputRequired(), NumberRange(0,
    100, "Please enter a reasonable number")])
    fta = DecimalField('Free Throws Attempted', validators=[InputRequired(), NumberRange
    (0, 100, "Please enter a reasonable number")])
    turnovers = DecimalField('Turnovers', validators=[InputRequired(), NumberRange(0,
    100, "Please enter a reasonable number")])
    prev = HiddenField()
    submit = SubmitField('Calculate')

    f = open("data.txt", "r")
    ptsVal = Decimal(f.readline())
    rbsVal = Decimal(f.readline())
    astVal = Decimal(f.readline())
    stlVal = Decimal(f.readline())
    blkVal = Decimal(f.readline())
    fgpVal = Decimal(f.readline())
    ftpVal = Decimal(f.readline())
    trnVal = Decimal(f.readline())
    thrVal = Decimal(f.readline())
    fgaVal = Decimal(f.readline())
    ftaVal = Decimal(f.readline())
    f.close()
```

app/routes.py

```
from flask import render_template, redirect, url_for
from app import app
from app.forms import IndexForm
from decimal import Decimal

@app.route('/')
def hello():
    return redirect(url_for('index'))

@app.route('/index', methods=['GET', 'POST'])
def index():
    form = IndexForm()
    if form.validate_on_submit():
        #Points computation
        total = form.points.data
        #Rebounds computation
        inter = form.ptsVal / form.rbsVal
        inter *= form.rebounds.data
        total += inter
        #Assists computation
        inter = form.ptsVal / form.astVal
        inter *= form.assists.data
        total += inter
        #Steals computation
        inter = form.ptsVal / form.stlVal
        inter *= form.steals.data
        total += inter
        #Blocks computation
        inter = form.ptsVal / form.blkVal
        inter *= form.blocks.data
        total += inter
        #Threes computation
        inter = form.ptsVal / form.thrVal
        inter *= form.threes.data
        total += inter
        #FGP computation
        if form.fga.data != 0:
            inter = form.fgm.data / form.fga.data
            inter -= form.fgpVal
            inter /= form.fgpVal
            inter *= form.fga.data
            inter *= form.ptsVal
            inter /= form.fgaVal
        else:
            inter = 0
        total += inter
        #FTP computation
        if form.fta.data != 0:
            inter = form.ftm.data / form.fta.data
            inter -= form.ftpVal
            inter /= form.ftpVal
            inter *= form.fta.data
            inter *= form.ptsVal
            inter /= form.ftaVal
        else:
            inter = 0
        total += inter
```

app/routes.py

```
#Turnovers computation
inter = form.ptsVal / form.trnVal
inter *= form.turnovers.data
total -= inter
#Display
if form.prev.data == "":
    form.prev.data = form.name.data + ": " + str(round(total, 2))
else:
    form.prev.data += ", " + form.name.data + ": " + str(round(total, 2))
li = form.prev.data.split(", ")
li.sort(reverse=True, key=myFunc)
return render_template('index.html', title='Results', form=form, process=True,
li=li)
return render_template('index.html', title='Home', form=form)

def myFunc(e):
    return Decimal(e[e.find(":")+2:len(e)+1])
```

app / templates / base.html

```
{% extends 'bootstrap/base.html' %}
```

```
{% block title %}
```

```
    {% if title %}Basketball Performance Metric{% else %}Welcome to the Basketball  
Performance Metric Program{% endif %}
```

```
{% endblock %}
```

```
{% block content %}{% endblock %}
```

app/templates/index.html

```
{% extends "base.html" %}

{% import 'bootstrap/wtf.html' as wtf %}

{% block content %}
  <h1 style="margin-left:14px">Player Statistics</h1>
  <div align="left">
    <div class="col-md-2">
      {{ wtf.quick_form(form) }}
    </div>
  </div>
  {% if process %}
    <br><br><br><br>
    <h1 align="center">
      <p>Results:</p>
      {% for result in li %}
        <p>{{result}}</p>
      {% endfor %}
    </h1>
  {% endif %}
  <br><br>
  <p align="center"></p>
  <br><br><br><br>
{% endblock %}
```